CENTRO UNIVERSITÁRIO UNIFECAF

CIÊNCIA DE DADOS

DENIS DORNELES VIANA

DO EVENTO AO INSIGHT

Construção e validação de um pipeline de Big Data para o e-commerce NovaMarket

TABOÃO DA SERRA — SP

2026

DENIS DORNELES VIANA

DO EVENTO AO INSIGHT

Construção e validação de um pipeline de Big Data para o e-commerce NovaMarket

Trabalho apresentado como requisito parcial de avaliação da disciplina Big Data and Distributed Computing, do curso de Ciência de Dados do Centro Universitário UniFECAF.

TABOÃO DA SERRA — SP

2026

SUMÁRIO

1. Sumário executivo

2. Diagnóstico do cenário atual

3. Os cinco Vs do Big Data

4. Arquitetura proposta e escolha da abordagem

5. Estratégia de ingestão com Apache Kafka

6. Processamento distribuído com Apache Spark

7. Streaming: eventos, janelas e watermark

8. Organização do Data Lake

9. Orquestração com Apache Airflow

10. Qualidade, rastreabilidade e monitoramento

11. Otimização de desempenho

12. Resultados e evidências da prova de conceito

13. Exemplo real pesquisado

14. Recomendações para produção

15. Principais riscos e limitações

16. Conclusão

17. Referências

18. Apêndice: arquitetura e evidências

As seções seguem o modelo acadêmico fornecido, com títulos numerados e subdivisões. A numeração de páginas pode variar conforme a edição do documento.

1. Sumário executivo

A NovaMarket é um cenário fictício de e-commerce utilizado para estudar os principais desafios de engenharia de dados em ambientes distribuídos. A situação descrita na atividade envolve arquivos CSV e JSON, processamento concentrado em um servidor, atrasos na atualização de indicadores, duplicidades, diferentes formatos, ausência de camadas de armazenamento e execução manual de rotinas.

Para responder ao problema foi implementada uma prova de conceito (POC) com gerador Python, Apache Kafka, Apache Spark Structured Streaming, Data Lake em camadas RAW, TRUSTED e REFINED, formato Parquet e Apache Airflow. O projeto foi executado no VS Code, com Ubuntu/WSL 2 e contêineres Docker.

A execução acompanhada em 9 de novembro de 2026 registrou 200 eventos de pedidos na RAW, confirmou arquivos Parquet nas camadas tratada e refinada e concluiu as quatro tarefas da DAG do Airflow. Em benchmark separado, com 300 mil registros sintéticos, o broadcast join reduziu o tempo medido de 1,7516 s para 0,4577 s, ou 73,87%, sem alterar o total calculado. Trata-se de um resultado pontual de laboratório, e não de uma estimativa de desempenho de toda a plataforma em produção.

2. Diagnóstico do cenário atual

2.1 Ambiente de dados

O enunciado caracteriza uma operação em crescimento, com dados de pedidos, navegação, estoque e campanhas gerados em diferentes velocidades e formatos. O uso de scripts centralizados cria um ponto de contenção, tanto para processar grandes volumes quanto para lidar com eventos produzidos continuamente.

2.2 Principais problemas

Problema do cenário	Efeito observado ou risco	Resposta adotada na POC
Arquivos CSV e JSON dispersos	Dificuldade de integração	Persistência em RAW e conversão para Parquet
Processamento em servidor único	Gargalos de execução	Spark distribuído com Master e Worker
Eventos atrasados e duplicados	Inconsistência das métricas	Event time, watermark e deduplicação
Pipelines executados manualmente	Falta de rastreabilidade	DAG Airflow com tarefas encadeadas
Joins e agregações lentos	Movimentação de dados (shuffle)	Comparação experimental com broadcast join

2.3 Objetivo do estudo

Construir e validar, em escala reduzida, uma solução que integre ingestão, processamento contínuo, preparação de dados, armazenamento analítico, orquestração e análise de

desempenho. A proposta não pretende reproduzir o volume de milhões de eventos do cenário empresarial, mas exercitar os mesmos conceitos técnicos de maneira verificável.

3. Os cinco Vs do Big Data

Dimensão	Interpretação no caso NovaMarket
Volume	A operação pode gerar muitos pedidos e interações. A POC utiliza 200 eventos e um conjunto sintético de 300 mil linhas para testar o processamento.
Velocidade	Os eventos são gerados continuamente e precisam ser capturados sem depender de uma exportação manual.
Variedade	Pedidos, campanhas, estoque e navegação podem apresentar esquemas distintos; o protótipo usa eventos JSON e saídas em Parquet.
Veracidade	Registros duplicados, preços inválidos, chaves nulas e horários inconsistentes podem prejudicar indicadores.
Valor	A finalidade é fornecer métricas de vendas por categoria e demonstrar decisões técnicas de otimização.

4. Arquitetura proposta e escolha da abordagem

4.1 Abordagem Kappa com processamento analítico complementar

A arquitetura toma como referência os princípios de Kappa porque a ingestão de eventos parte de um log persistente do Kafka e o Spark Structured Streaming constitui a entrada contínua dos dados. Há também um job batch downstream que lê a RAW para construir as camadas analíticas. Portanto, trata-se de uma POC inspirada em Kappa, mas não de uma implementação Kappa pura: o fluxo inclui materialização e reprocessamento em lote após a etapa de streaming.

Essa escolha é adequada ao objetivo didático. O Kafka desacopla a produção do consumo e permite organizar eventos por partições; o Spark processa os eventos, enquanto os arquivos RAW preservam os registros recebidos para nova transformação posterior. A alternativa Lambda, com duas lógicas paralelas completas para batch e velocidade, acrescentaria complexidade desnecessária nesta escala.

4.2 Arquitetura lógica

```
PRODUTOR PYTHON → KAFKA (3 PARTIÇÕES)
                        ↓
              SPARK STRUCTURED STREAMING
                        ↓
RAW (JSON) → TRUSTED (PARQUET) → REFINED (PARQUET)
                        ↑
                  SPARK BATCH

AIRFLOW: CHECK_INPUT → PROCESS_DATA → QUALITY_CHECK → BENCHMARK
```

Figura 1 — Arquitetura lógica simplificada da prova de conceito.

4.3 Stack de tecnologias

Camada	Tecnologia	Função
Ingestão	Python + Kafka 3.9.1	Produção e retenção de eventos em tópico
Processamento	Spark 3.5.5	Streaming, validação, agregação e benchmark
Persistência	Data Lake local	JSON na RAW e Parquet nas camadas analíticas
Orquestração	Airflow 2.10.5	Execução e controle de dependências da DAG
Ambiente	Docker Desktop + WSL 2	Contêineres isolados gerenciados pelo VS Code

5. Estratégia de ingestão com Apache Kafka

O tópico novamarket-pedidos foi configurado com três partições e fator de replicação igual a um, suficiente para um laboratório de broker único. O produtor Python gera mensagens JSON com os atributos id_pedido, id_cliente, categoria, quantidade, valor_unitario, campanha e event_time. A chave da mensagem é o id_pedido, favorecendo a distribuição determinística de registros com o mesmo identificador para uma mesma partição quando se mantém o particionador.

5.1 Evento de exemplo e características

```
{ "id_pedido": "PED-00000024", "categoria": "eletronicos",
"quantidade": 3, "valor_unitario": 1356.35,
"campanha": "black_friday", "event_time": "2026-11-09T15:44:50+00:00" }
```

Registro ilustrativo resumido a partir do formato observado na RAW; outros campos foram omitidos para facilitar a leitura.

O gerador emprega acks=all e realiza flush ao finalizar. No ambiente testado foram enviados 200 eventos e, depois, encontrados 200 registros JSON na RAW. A replicação 1 não oferece tolerância a perda do único broker.

5.2 Decisões de particionamento

As três partições possibilitam paralelismo de consumo e evitam concentrar todos os registros em um único log. Como a POC é pequena, não foi executado um teste formal para determinar o número ideal de partições. Em produção, a escolha dependeria de throughput, número de consumidores, distribuição das chaves e capacidade de recuperação.

6. Processamento distribuído com Apache Spark

O processamento em lote lê os JSON da RAW segundo um schema predefinido. A transformação converte event_time em timestamp, calcula valor_total como quantidade multiplicada por valor_unitario e aplica filtros de validade: identificador e categoria presentes, quantidade positiva, preço não negativo e data de evento interpretável.

Na camada TRUSTED, o job utiliza dropDuplicates por id_pedido. Na REFINED, agrupa por categoria e calcula quantidade de pedidos, faturamento total e ticket médio. O job contém explain(True), permitindo observar o plano de execução de consultas. Esse recurso ajuda a entender agregações, exchanges e movimentos de dados, embora o relatório não disponha de uma exportação detalhada do plano físico dessa execução.

6.1 Indicadores produzidos

Indicador	Definição técnica
Pedidos por categoria	COUNT(*) após filtragem e deduplicação.
Faturamento por categoria	SUM(valor_total), com arredondamento em duas casas.
Ticket médio	AVG(valor_total), com arredondamento em duas casas.

7. Streaming: eventos, janelas e watermark

O Spark Structured Streaming consome o tópico Kafka e interpreta o conteúdo JSON segundo o schema dos pedidos. A consulta grava os eventos brutos em arquivos JSON na RAW e mantém checkpoints. Em uma segunda consulta, trata os registros e calcula eventos e faturamento por categoria em janelas de cinco minutos, utilizando watermark de dez minutos sobre o instante do evento.

7.1 Event time e processing time

Event time representa o instante em que um pedido ocorreu na origem, registrado em event_time. Processing time é o momento em que o evento é recebido e processado pelo Spark. Esses momentos podem ser diferentes quando há atrasos de rede, filas ou indisponibilidade temporária de consumidores.

O gerador inclui registros com event_time quatro minutos anterior ao instante de envio a cada 25 eventos, permitindo experimentar atrasos. O watermark de dez minutos é um mecanismo para limitar o estado mantido por agregações temporais e controlar até quando janelas ainda podem

ser atualizadas. A configuração não garante que qualquer evento atrasado seja aceito independentemente do estado do watermark no momento do processamento.

8. Organização do Data Lake

8.1 Camada RAW

A RAW preserva os registros JSON ingeridos do Kafka e constitui o ponto inicial para auditoria e reprocessamento. Na execução observada foram identificados três arquivos part-*.json e 200 linhas no total. Os diretórios _spark_metadata e _checkpoints guardam informações técnicas relativas ao streaming.

8.2 Camada TRUSTED

A TRUSTED recebe os registros filtrados, normalizados e deduplicados. Os arquivos são gravados em Parquet com compressão Snappy. Nessa camada, os dados passam a ter tipos e colunas apropriados ao processamento analítico.

8.3 Camada REFINED

A REFINED armazena os indicadores agregados de vendas por categoria, também em Parquet. O armazenamento colunar pode reduzir leituras de colunas desnecessárias, melhorar compressão e facilitar consultas analíticas em comparação com JSON texto. A execução não mediu diretamente a diferença de tempo entre JSON e Parquet; essa vantagem é conceitual e não deve ser apresentada como resultado experimental próprio.

9. Orquestração com Apache Airflow

A DAG novamarket_pipeline foi configurada com quatro tarefas encadeadas. A primeira verifica a existência da entrada, a segunda submete o processamento batch, a terceira verifica os artefatos esperados e a quarta executa o benchmark. A captura do Airflow enviada após as correções mostrou as quatro tarefas em estado success, com execução iniciada em 9 de novembro de 2026 às 16h51min33s UTC.

```
check_input → process_data → quality_check → benchmark
```

Uma incompatibilidade entre o Python 3.12 do contêiner Airflow e o Python 3.8 do Worker provocou falhas na execução distribuída. A correção final passou a submeter os jobs a partir do contêiner Spark Master, onde Driver e Worker utilizam a mesma versão principal e secundária do Python. O Airflow passa a atuar apenas como orquestrador da execução.

10. Qualidade, rastreabilidade e monitoramento

As regras implementadas verificam campos obrigatórios, quantidades positivas, preços não negativos e timestamps válidos. A deduplicação por id_pedido evita múltiplas linhas com a mesma chave na TRUSTED, embora, se dois registros divergentes tiverem o mesmo identificador, a versão preservada não esteja definida por uma regra de negócio explícita.

Dimensão	Regra implementada / evidência
Completude	id_pedido e categoria não nulos
Validade	quantidade maior que zero e valor_unitario não negativo
Temporalidade	event_time convertido para data_evento válida
Unicidade	dropDuplicates por id_pedido
Execução	Airflow Graph e logs de tarefas
Estado do stream	Checkpoints, offsets e microbatches do Spark

O monitoramento na POC foi feito pelas interfaces Spark Master e Airflow e pelos logs do terminal. Não foram implantados alertas automatizados, dashboards de SLA ou catálogo de lineage.

11. Otimização de desempenho

11.1 Gargalo e hipótese

A movimentação de dados em joins é um custo relevante no Spark. Um join convencional pode exigir redistribuição de linhas entre partições, enquanto um broadcast join permite disponibilizar uma dimensão pequena aos executores. A hipótese testada foi que transmitir uma tabela de quatro categorias aos executores seria mais eficiente do que executar o join convencional no cenário sintético proposto.

11.2 Método experimental

O benchmark gerou 300 mil linhas sintéticas, materializou o DataFrame principal em cache e comparou a junção padrão com broadcast automático desabilitado versus a junção explicitamente assistida por broadcast(categorias). Ambos os cenários calcularam soma por categoria e retornaram os resultados ao Driver. O indicador medido foi o tempo de execução em segundos.

11.3 Comparação antes e depois

Aspecto	Antes	Alteração	Depois
Estratégia de join	Join convencional	Broadcast explícito da dimensão	Broadcast join
Tempo medido	1,7516 s	Dimensão pequena enviada aos executores	0,4577 s
Categorias	4	Sem mudança na regra de negócio	4
Total calculado	149.850.000	Sem mudança na agregação	149.850.000

A redução de tempo observada foi de 73,87%: [(1,7516 − 0,4577) / 1,7516] × 100. O broadcast foi cerca de 3,83 vezes mais rápido nesta execução. Em uma medição anterior, foram

registrados 1,6094 s e 0,4603 s, com redução de 71,40%. Os dois resultados são favoráveis, mas não substituem uma bateria estatística de testes sob diferentes condições de carga.

11.4 Interpretação

O resultado apoia a hipótese de que broadcast join pode reduzir o custo de uma junção com dimensão pequena. Contudo, não comprova sozinho que o ganho ocorreu exclusivamente por redução de shuffle, pois não foi apresentada medição isolada do volume de shuffle. Também não significa que o pipeline de ponta a ponta tenha melhorado 73,87%.

12. Resultados e evidências da prova de conceito

Componente	Resultado verificado	Tipo de evidência
Kafka	Tópico com 3 partições	Descrição do tópico no terminal
RAW	200 eventos em arquivos JSON	Contagem de linhas e amostra do conteúdo
TRUSTED	Arquivo Parquet criado	Listagem de arquivos
REFINED	Arquivo Parquet criado	Listagem de arquivos
Airflow	4 tarefas verdes	Captura da aba Graph
Benchmark	Redução de 73,87%	JSON de resultados da execução

As evidências foram coletadas no ambiente local e apresentadas por meio de capturas de tela durante a execução. Recomenda-se anexar ao PDF final a captura da DAG com as quatro tarefas verdes e a saída do JSON do benchmark; o relatório não reproduz imagens não disponibilizadas como arquivos de figura editáveis.

13. Exemplo real pesquisado

Um exemplo externo de uso de arquitetura orientada a eventos é a Uber. Um relato técnico publicado no blog do Apache Hudi em dezembro de 2025 descreve uma evolução da ingestão do Data Lake de batch para streaming, com eventos recebidos pelo Kafka e processados por Apache Flink, gravando dados em formato Apache Hudi. O estudo menciona desafios de arquivos pequenos, desequilíbrio de partições e sincronização de checkpoints.

A referência contribui para o NovaMarket ao ilustrar por que uma operação real precisa ir além de “consumir e gravar”: decisões sobre frescor, escalabilidade, qualidade e recuperação têm impacto operacional. O caso citado usa Flink e Hudi, não a mesma combinação de Spark e Parquet da POC, e seus resultados não foram reproduzidos neste trabalho.

14. Recomendações para produção

- Utilizar Kafka com múltiplos brokers, replicação adequada, controle de retenção, monitoramento de lag e políticas de segurança.
- Definir contratos de schema, regras de compatibilidade, identificadores idempotentes e política de atualização dos pedidos.

- Separar ambientes de desenvolvimento, homologação e produção; usar orquestração sem acesso privilegiado ao Docker socket.
- Avaliar Delta Lake ou formato transacional equivalente para garantir maior controle de escrita, atualização e reprocessamento.
- Executar testes de carga com volumes maiores, chaves desbalanceadas, eventos realmente tardios e falhas simuladas.
- Monitorar throughput, freshness, latência, tamanho dos arquivos, volume de shuffle e falhas por etapa.
- Testar broadcast com dimensões de tamanhos diferentes e impor limites para evitar pressão de memória.

15. Principais riscos e limitações

Risco/limitação	Impacto	Mitigação sugerida
Broker único	Ausência de tolerância a falhas	Cluster replicado de Kafka
Arquivos Parquet locais	Baixa robustez transacional	Armazenamento e tabelas com ACID
Docker socket no Airflow	Acesso privilegiado ao host	Executor remoto com autenticação
Volume reduzido de testes	Representatividade limitada	Testes com escalas e cargas variadas
Um benchmark pontual	Variação de tempos	Múltiplas rodadas e análise de dispersão
Deduplicação sem ordem de escolha	Possível ambiguidade em eventos divergentes	Regra determinística por versão e horário

16. Conclusão

O projeto NovaMarket demonstrou a integração de mecanismos fundamentais de Big Data: ingestão desacoplada com Kafka, processamento distribuído e contínuo com Spark, armazenamento em camadas e orquestração no Airflow. A execução acompanhada confirmou a chegada de 200 registros à RAW, a geração das saídas Parquet e a conclusão de uma DAG de quatro tarefas.

No teste de otimização, o broadcast join reduziu em 73,87% o tempo medido em relação ao join convencional para uma base sintética de 300 mil registros, preservando quatro categorias e o total calculado. O resultado evidencia o valor de analisar o tipo e o tamanho das relações antes de escolher uma estratégia de junção.

Os incidentes encontrados durante a implantação — permissões de arquivos, recursos do cluster, armazenamento Docker e incompatibilidade de versões do Python — também foram parte relevante do aprendizado, pois demonstram que o desempenho e a confiabilidade dependem tanto da lógica de transformação quanto da correta configuração do ambiente distribuído.

17. Referências

APACHE SOFTWARE FOUNDATION. Apache Spark Documentation. https://spark.apache.org/docs/3.5.5/ . Acesso em novembro de 2026.

APACHE SOFTWARE FOUNDATION. Structured Streaming Programming Guide. https://spark.apache.org/docs/3.5.5/structured-streaming-programming-guide.html . Acesso em novembro de 2026.

APACHE SOFTWARE FOUNDATION. Apache Kafka Documentation. https://kafka.apache.org/documentation/ . Acesso em novembro de 2026.

APACHE SOFTWARE FOUNDATION. Apache Airflow Documentation. https://airflow.apache.org/docs/ . Acesso em novembro de 2026.

APACHE PARQUET. Documentation. https://parquet.apache.org/docs/ . Acesso em novembro de 2026.

APACHE HUDI BLOG. From Batch to Streaming: Accelerating Data Freshness in Uber's Data Lake. 11 dez. 2025. https://hudi.incubator.apache.org/blog/2025/12/12/from-batch-to-streaming-accelerating-data-freshness-in-ubers-data-lake/ .

DORNELES VIANA, Denis. NovaMarket — Do Evento ao Insight. Repositório de código. https://github.com/denisddv/trabalhocienciadedados . 2026.

18. Apêndice: arquitetura e evidências

18.1 Organização de diretórios

```
producer/              gerador de eventos Kafka
spark/                 streaming.py, batch.py, benchmark.py
airflow/dags/          novamarket.py
datalake/raw/          eventos em JSON
datalake/trusted/      registros tratados em Parquet
datalake/refined/      indicadores em Parquet
benchmark/             resultados.json (gerado localmente)
docs/                  documentação técnica
```

18.2 Arquivos e telas a anexar na entrega

- Captura do Kafka com três partições e fator de replicação igual a um.
- Captura ou comando comprovando 200 linhas na RAW.
- Captura Spark Master com o Worker registrado.
- Captura do Airflow com check_input, process_data, quality_check e benchmark em verde.
- Captura do benchmark JSON da última medição, com 73,87%.

Repositório do projeto: https://github.com/denisddv/trabalhocienciadedados

19. Vídeo de apresentação e evidências

O vídeo apresenta o pipeline de Big Data do projeto NovaMarket, incluindo Kafka, Spark Structured Streaming, Data Lake RAW/TRUSTED/REFINED, DAG do Airflow e benchmark de otimização.

Acesse o vídeo demonstrativo:

https://youtu.be/685Qp5Ovofg

Aponte a câmera para abrir o vídeo.

Material complementar

As capturas da execução estão organizadas no Anexo de Evidências: Kafka, Spark, streaming, arquivos Parquet, Airflow com quatro tarefas verdes e resultados do benchmark. O código e a documentação estão no repositório GitHub.

github.com/denisddv/trabalhocienciadedados